



**UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA**  
**CARRERA DE INGENIERÍA DE SOFTWARE**

**ASIGNATURA: INTELIGENCIA DE NEGOCIOS**

**ENTREGABLE 2: CONSTRUCCIÓN DEL DATA MART**

De un dataset real a una estructura analítica en PostgreSQL

Dataset analizado:



**ESTUDIANTES**

**Diana Lucía Melena Santander**

SANTA ELENA – ECUADOR  
2026

# CONSTRUCCIÓN DEL DATA MART

## 1. Arquitectura y archivos entregados

La solución sigue el flujo pedido: dataset de Kaggle, limpieza en Python, carga en PostgreSQL, modelo dimensional, vistas y funciones, y finalmente la aplicación analítica. Los archivos que acompañan a este documento son los siguientes:

| Archivo                      | Contenido                                                                                                             |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| docker-compose.yml           | Definición del contenedor de PostgreSQL 17 y del volumen que expone la carpeta del proyecto dentro del contenedor.    |
| 01_modelo.sql                | Creación del esquema dm, las seis dimensiones, las dos tablas de hechos, claves primarias, claves foráneas e índices. |
| 02_limpieza.py               | Lectura del archivo Excel de Kaggle, limpieza, transformaciones y generación de los archivos de carga.                |
| 03_carga.sql                 | Tablas de staging, carga con \copy y poblado de dimensiones y hechos.                                                 |
| 04_validaciones.sql          | Consultas de conteo, integridad referencial, nulos y totales.                                                         |
| 05_vistas_funciones.sql      | Cuatro vistas y tres funciones que alimentan el dashboard.                                                            |
| assets/diagrama_estrella.svg | Diagrama del modelo dimensional.                                                                                      |
| dashboard/                   | Mockup navegable del dashboard, separado en index.html, styles.css y app.js.                                          |

**Repositorio:** <https://github.com/Misstery13/bi-lol-datamart>

**Dashboard publicado (GitHub Pages):** <https://misstery13.github.io/bi-lol-datamart/dashboard/>

## 2. Configuración de PostgreSQL con Docker

Se levantó PostgreSQL 17 en un contenedor de Docker en lugar de instalarlo en el sistema operativo. De esta forma el entorno queda aislado y es reproducible en cualquier equipo del grupo, y la base puede iniciarse, detenerse o reconstruirse sin afectar al sistema anfitrión. La definición está en docker-compose.yml:

```
services:
  postgres:
    image: postgres:17
    container_name: bi-postgres
    restart: unless-stopped
    environment:
      POSTGRES_USER: bi_user
      POSTGRES_PASSWORD: bi_password
      POSTGRES_DB: bi_database
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./:/proyecto

volumes:
  postgres_data:
```

Al volumen de datos se añadió un segundo volumen que monta la carpeta del proyecto dentro del contenedor como /proyecto. Sin él, psql no podría leer los scripts ni los archivos CSV generados por la limpieza, porque el contenedor no ve el sistema de archivos del equipo.

Los datos de conexión resultantes son: host localhost, puerto 5432, base bi\_database, usuario bi\_user y contraseña bi\_password. La secuencia completa de ejecución es:

```

docker compose up -d
python 02_limpieza.py
docker compose exec postgres psql -U bi_user -d bi_database -f /proyecto/01_modelo.sql
docker compose exec postgres psql -U bi_user -d bi_database -f /proyecto/03_carga.sql
docker compose exec postgres psql -U bi_user -d bi_database -f /proyecto/04_validaciones.sql
docker compose exec postgres psql -U bi_user -d bi_database -f
/proyecto/05_vistas_funciones.sql

# para apagar el entorno
docker compose down

```

La limpieza se ejecuta en el equipo anfitrión y no dentro del contenedor, porque la imagen de PostgreSQL no incluye Python ni pandas. Sus salidas quedan en ./salida, que el contenedor ve como /proyecto/salida.

Los scripts se ejecutan con psql y no con pgAdmin porque el archivo de carga utiliza el comando \copy, que es una instrucción propia del cliente psql y no una sentencia SQL estándar. Los datos persisten en el volumen postgres\_data, de modo que docker compose down no borra la base; para empezar desde cero habría que usar docker compose down -v.

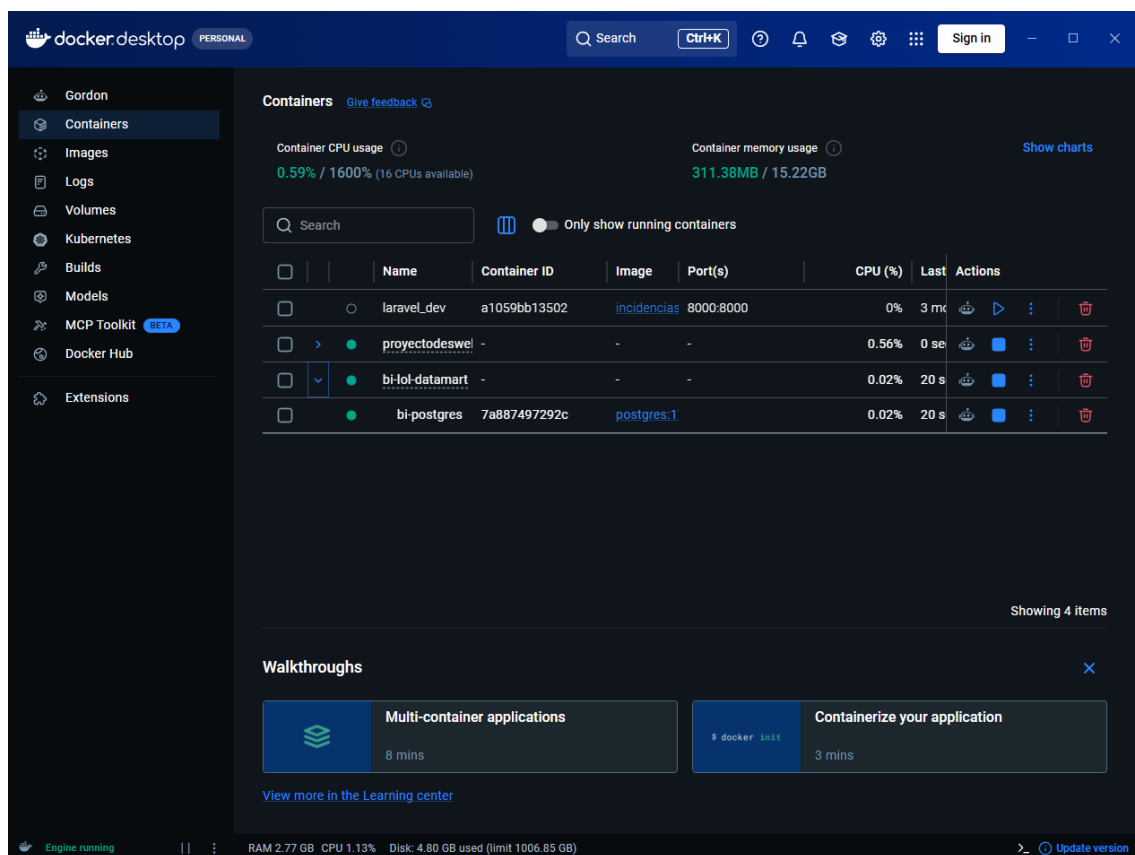


Figura 1. Contenedor bi-postgres en ejecución dentro del proyecto bi-lol-datamart en Docker Desktop.

### 3. Transformaciones aplicadas al dataset original

| Problema detectado                                                                                | Transformación aplicada                                                                                                                                 |
|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Granularidad mezclada: cada partida trae doce filas, diez de jugador y dos de resumen por equipo. | Separación en dos conjuntos mediante la columna position. Las filas de equipo alimentan el hecho principal; las de jugador alimentan el hecho de draft. |
| La columna date se importa como texto.                                                            | Conversión a fecha y hora, y derivación de la columna fecha para enlazar con la dimensión de tiempo.                                                    |
| La columna patch se lee como número y pierde su formato.                                          | Conversión a texto y descomposición en temporada y número de parche.                                                                                    |

| Problema detectado                                                    | Transformación aplicada                                                                                                           |
|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Partidas con duración muy baja (remakes por desconexión).             | Exclusión de las partidas de menos de quince minutos, por ser valores atípicos que distorsionan los promedios.                    |
| Nombres de liga, split y equipo con espacios y variantes.             | Normalización de espacios y reemplazo de los split vacíos por la etiqueta Sin split.                                              |
| Columnas de corte temporal vacías cuando datacompleteness es parcial. | Se conserva el nulo de forma deliberada en oro_dif_15, porque distinguir el dato ausente del valor cero es necesario para el KPI. |
| Columnas de objetivos con nulos en ligas menores.                     | Relleno con cero, ya que la ausencia de registro equivale a no haber conseguido el objetivo.                                      |
| Columnas sin aporte analítico (url, identificadores internos).        | Descartadas en la selección final de columnas.                                                                                    |
| Posibles duplicados por partida y equipo.                             | Eliminación de duplicados sobre la combinación gameid más equipo.                                                                 |

La ejecución de 02\_limpieza.py sobre el archivo descargado de Kaggle produjo el siguiente resumen. El archivo cubre las partidas jugadas entre el 1 de enero y el 13 de febrero de 2024: de 12 276 filas originales resultan 1 023 partidas únicas, 2 046 registros de equipo por partida y 19 658 registros de draft.

```
Windows PowerShell

PS C:\Users\DIANA MAIN\Documents\GitHub\bi-lol-datamart> python 02_limpieza.py
Filas originales: 12,276 | Columnas: 123
Filas de equipo: 2,046 | Filas de jugador: 10,230
Remakes descartados (menos de 15 min): 0
Duplicados eliminados: 0
Registros sin oro_dif_15 (datos parciales): 374 (18.3%)

--- RESUMEN ---
Partidas (equipo-partida): 2,046
Partidas unicas:          1,023
Registros de draft:       19,658
Días en dim_tiempo:       36
Rango de fechas:         2024-01-01 a 2024-02-13
Archivos generados en ./salida/
```

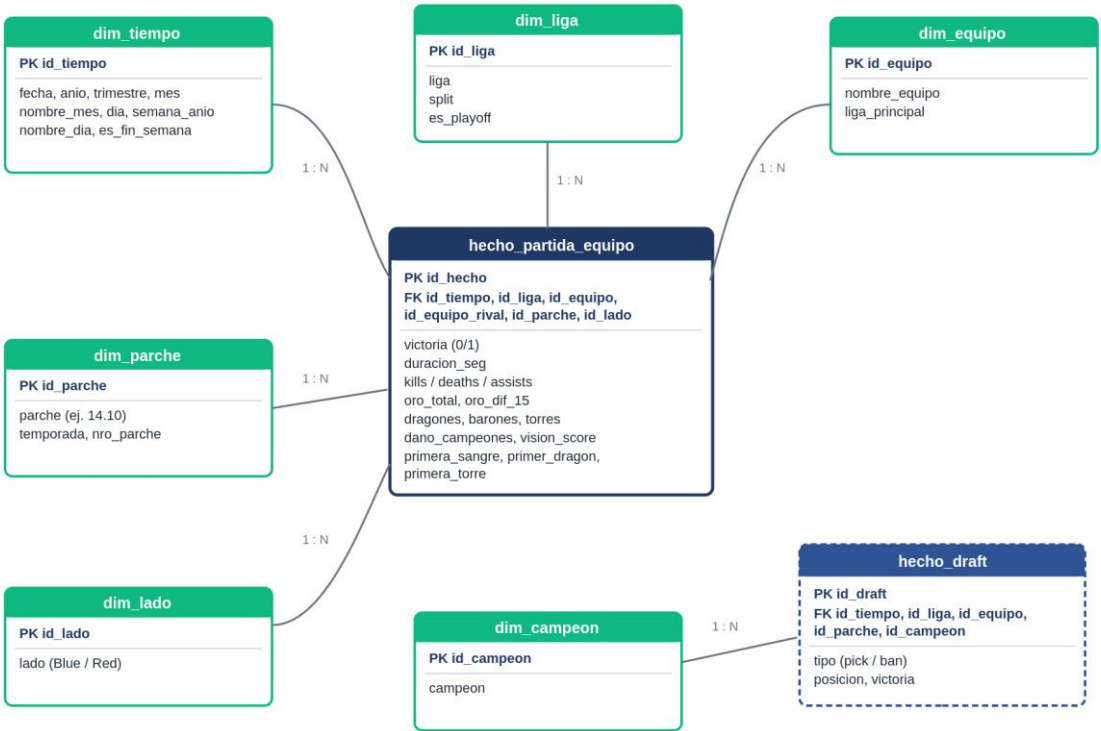
Figura 2. Salida de 02\_limpieza.py con los datos reales.

## 4. Modelo dimensional y granularidad

**Granularidad de la tabla de hechos:** un registro de hecho\_partida\_equipo representa la participación de un equipo en una partida competitiva. Cada partida genera, por lo tanto, exactamente dos registros.

Se eligió este nivel y no el de jugador porque los tres KPI definidos en el entregable anterior se calculan sobre resultados de equipo. Para el KPI de presencia en el draft, que sí necesita el detalle de campeón, se incorporó una segunda tabla de hechos, hecho\_draft, cuya granularidad es un campeón seleccionado o baneado por un equipo en una partida. Ambas comparten las mismas dimensiones.

Modelo dimensional - Data Mart de partidas competitivas de League of Legends



Granularidad del hecho principal: un registro representa la participacion de un equipo en una partida competitiva (dos registros por partida).  
El hecho secundario hecho\_draft baja a nivel de campeon y comparte las mismas dimensiones.

Figura 3. Diagrama estrella del Data Mart.

| Objeto               | Tipo      | Clave primaria       | Observación                                                      |
|----------------------|-----------|----------------------|------------------------------------------------------------------|
| dim_tiempo           | Dimensión | id_tiempo (AAAAMMDD) | Un registro por día del calendario competitivo.                  |
| dim_liga             | Dimensión | id_liga              | Combina liga, split y si es fase de playoffs.                    |
| dim_equipo           | Dimensión | id_equipo            | Incluye la liga en la que el equipo disputó más partidas.        |
| dim_parche           | Dimensión | id_parche            | Versión del juego; es una dimensión temporal propia del dominio. |
| dim_lado             | Dimensión | id_lado              | Lado azul o rojo del mapa.                                       |
| dim_campeon          | Dimensión | id_campeon           | Alimenta el KPI de presencia en el draft.                        |
| hecho_partida_equipo | Hechos    | id_hecho             | Seis claves foráneas y quince medidas.                           |
| hecho_draft          | Hechos    | id_draft             | Cinco claves foráneas; distingue picks de bans.                  |

5. Medidas asociadas a cada KPI

| KPI                            | Medidas que lo sostienen                                                                     | Objeto SQL                                 |
|--------------------------------|----------------------------------------------------------------------------------------------|--------------------------------------------|
| Tasa de victoria               | victoria, sobre el conteo de partidas, agrupado por equipo, liga y split.                    | vw_kpi_equipo, fn_tasa_victoria()          |
| Presencia en el draft          | Conteo de partidas en que aparece el campeón, dividido para el total de partidas del parche. | vw_presencia_campeon, fn_top_campeones()   |
| Diferencia de oro al minuto 15 | Promedio de oro_dif_15, excluyendo los registros con dato parcial.                           | vw_ventaja_temprana, fn_evolucion_equipo() |

## 6. Consultas de validación

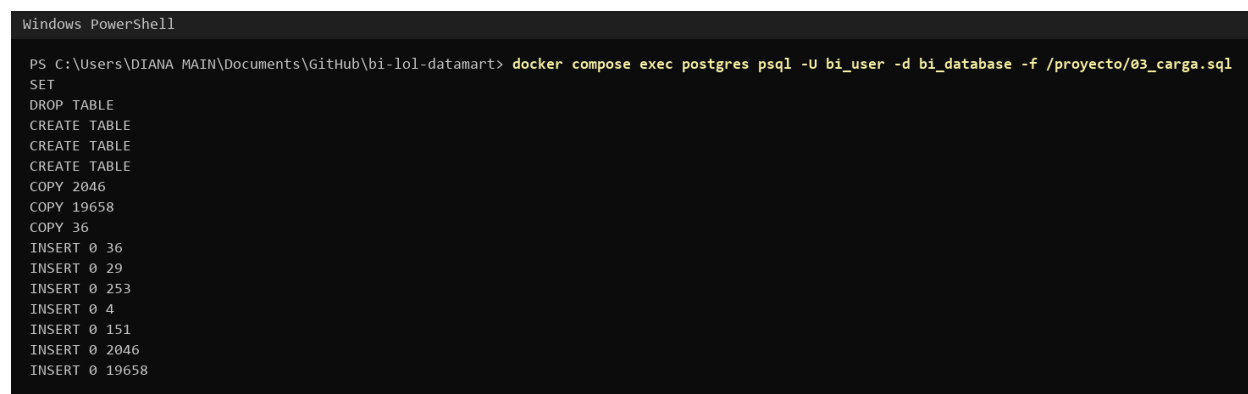
El archivo 04\_validaciones.sql agrupa cinco bloques. Los resultados esperados son los siguientes:

- V1, conteo por tabla: debe haber dos registros de hecho por cada partida única y todas las dimensiones deben tener filas.
- V2, granularidad: la consulta que busca partidas con un número de equipos distinto de dos debe devolver cero filas, igual que la que busca partidas sin exactamente un ganador.
- V3, integridad referencial: los conteos de hechos huérfanos deben ser cero. También se revisan las dimensiones sin uso, que señalarían filas perdidas durante la carga.
- V4, nulos relevantes: se mide el porcentaje de registros sin oro\_dif\_15 y se identifica qué ligas los concentran. Este nulo es esperado y documentado.
- V5, totales: la tasa de victoria global debe ser exactamente cincuenta por ciento, porque cada partida tiene un ganador y un perdedor. Si no lo es, hay partidas incompletas en el hecho. También se comprueba la conocida ventaja del lado azul, como validación de sentido del dominio.

La validación V5 es la más útil de todas: un error de carga que duplique o pierda filas se detecta de inmediato, porque la tasa global deja de ser del cincuenta por ciento.

Al ejecutar los scripts sobre los datos reales se obtuvieron los siguientes resultados, que confirman una carga correcta.

Carga: los tres archivos de staging se cargaron completos (2 046, 19 658 y 36 filas) y el poblado del modelo insertó 36 días, 29 ligas, 253 equipos, 4 parches, 151 campeones, 2 046 hechos de partida y 19 658 hechos de draft.



```
Windows PowerShell

PS C:\Users\DIANA MAIN\Documents\GitHub\bi-lol-datamart> docker compose exec postgres psql -U bi_user -d bi_database -f /proyecto/03_carga.sql
SET
DROP TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
COPY 2046
COPY 19658
COPY 36
INSERT 0 36
INSERT 0 29
INSERT 0 253
INSERT 0 4
INSERT 0 151
INSERT 0 2046
INSERT 0 19658
```

*Figura 4. Evidencia de carga: salida de 03\_carga.sql.*

V1 a V3: hay exactamente dos registros de hecho por partida (1 023 partidas, 2 046 filas), ninguna partida con un número de equipos o de ganadores distinto del esperado, y cero hechos huérfanos de tiempo, equipo o liga.

```
Windows PowerShell

PS C:\Users\DIANA MAIN\Documents\GitHub\bi-lol-datamart> docker compose exec postgres psql -U bi_user -d bi_database -f /proyecto/04_validaciones.sql
SET
      tabla      | registros
-----+-----
 dim_campeon     |      151
 dim_equipo      |      253
 dim_lado        |         2
 dim_liga        |        29
 dim_parche      |         4
 dim_tiempo      |        36
 hecho_draft     |     19658
 hecho_partida_equipo |     2046
(8 rows)

partidas_unicas | filas_hecho | filas_por_partida
-----+-----+-----
          1023 |         2046 |          2.00
(1 row)

gameid | equipos
-----+-----
(0 rows)

gameid | ganadores
-----+-----
(0 rows)

hechos_sin_tiempo
-----+-----
              0
(1 row)

hechos_sin_equipo
-----+-----
              0
(1 row)

hechos_sin_liga
-----+-----
              0
(1 row)

nombre_equipo
-----+-----
(0 rows)
```

Figura 5. Validaciones de conteo, granularidad e integridad referencial.

V4 y V5: el 18,28 % de los registros no tiene oro\_dif\_15, y todos pertenecen a LPL, LDL y DCup, ligas que se publican con datos parciales. La tasa de victoria global es exactamente 50,00 % y la diferencia de oro promedio al minuto 15 es 0, como corresponde a una medida de suma cero entre los dos equipos de cada partida.

```
Windows PowerShell

-- (continuacion de la salida de 04_validaciones.sql)

total | con_oro_dif_15 | sin_oro_dif_15 | pct_nulos | sin_vision_score | sin_rival
-----+-----+-----+-----+-----+-----
2046 | 1672 | 374 | 18.28 | 0 | 0
(1 row)

liga | partidas | sin_oro_dif_15
-----+-----+-----
LPL | 190 | 190
LDL | 148 | 148
DCup | 36 | 36
(3 rows)

partidas | duracion_media_min | kills_totales | oro_promedio | oro_dif_15_promedio | tasa_victoria_global
-----+-----+-----+-----+-----+-----
1023 | 32.2 | 27993 | 58830 | 0 | 50.00
(1 row)

lado | partidas | tasa_victoria
-----+-----+-----
Red | 1023 | 45.45
Blue | 1023 | 54.55
(2 rows)

desde | hasta | dias
-----+-----+-----
2024-01-01 | 2024-02-13 | 36
(1 row)
```

Figura 6. Validaciones de nulos relevantes y totales.

Por último, las vistas y funciones de 05\_vistas\_funciones.sql devuelven datos reales y quedan listas para alimentar el dashboard.



```
Windows PowerShell

PS C:\Users\DIANA MAIN\Documents\GitHub\bi-lol-datamart> docker compose exec postgres psql -U bi_user -d bi_database
SET
-- Vista vw_kpi_equipo (LCK, top 6 por tasa de victoria)
-- equipo | liga | split | partidas | victorias | tasa_victoria | oro_dif_15_prom | duracion_media_min | dragones_prom | barones_prom
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
Gen.G | LCK | Spring | 14 | 12 | 85.71 | 1362 | 31.1 | 3.21 | 0.93
T1 | LCK | Spring | 14 | 11 | 78.57 | 2187 | 29.6 | 2.36 | 1.00
Hanwha Life Esports | LCK | Spring | 13 | 10 | 76.92 | 416 | 30.3 | 2.46 | 0.69
KT Rolster | LCK | Spring | 16 | 10 | 62.50 | 178 | 33.1 | 2.63 | 0.69
Kwangdong Freecs | LCK | Spring | 13 | 6 | 46.15 | -609 | 33.3 | 2.62 | 0.85
FearX | LCK | Spring | 14 | 6 | 42.86 | -540 | 31.2 | 2.07 | 0.71
(6 rows)

-- Funcion fn_tasa_victoria('LCK', '2024-01-01', '2024-02-13')
-- equipo | partidas | victorias | tasa_victoria
-----+-----+-----+-----
Gen.G | 14 | 12 | 85.71
T1 | 14 | 11 | 78.57
Hanwha Life Esports | 13 | 10 | 76.92
KT Rolster | 16 | 10 | 62.50
Kwangdong Freecs | 13 | 6 | 46.15
FearX | 14 | 6 | 42.86
(6 rows)

-- Funcion fn_top_campeones('14.02', 6)
campeon | presencia_pct | picks | bans | win_rate
-----+-----+-----+-----+-----
Karma | 85.86 | 31 | 230 | 54.84
Varus | 84.54 | 152 | 105 | 48.03
K'Sante | 79.28 | 164 | 77 | 56.71
Ashe | 78.95 | 49 | 191 | 59.18
Kalista | 78.62 | 88 | 151 | 53.41
Azir | 73.68 | 136 | 88 | 52.21
(6 rows)

-- Funcion fn_evolucion_equipo('T1')
anio | mes | nombre_mes | partidas | tasa_victoria | oro_dif_15_prom
-----+-----+-----+-----+-----+-----
2024 | 1 | enero | 10 | 70.00 | 1813
2024 | 2 | febrero | 4 | 100.00 | 3122
(2 rows)
```

Figura 7. Consulta de vistas y funciones con los datos cargados.

7. Mockup del dashboard

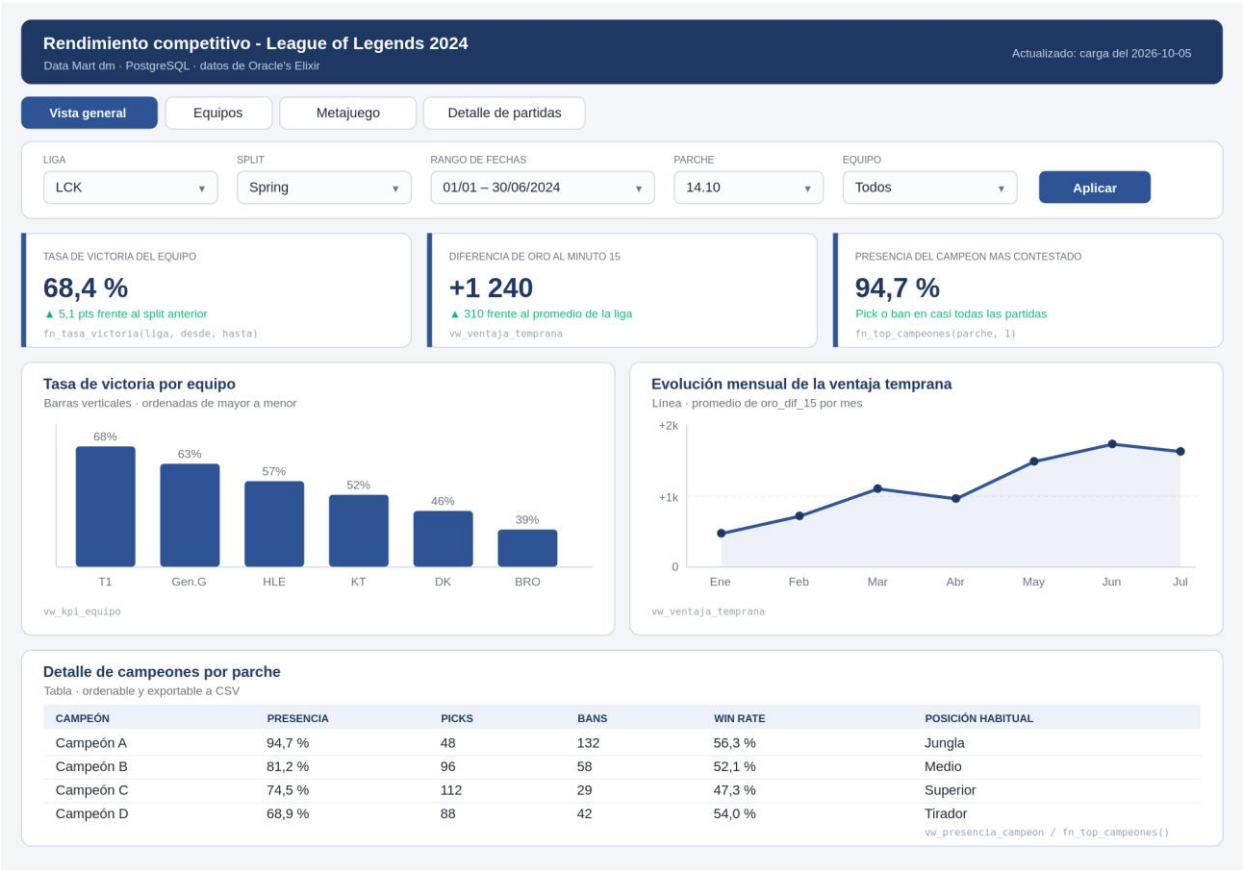


Figura 8. Mockup de la vista general del dashboard.

La aplicación se organiza en cuatro vistas. Vista general reúne los tres KPI, los dos gráficos y la tabla de detalle. Equipos profundiza en el rendimiento individual de cada equipo con su evolución mensual. Metajuego concentra el análisis de campeones por parche. Detalle de partidas permite llegar al registro individual para auditar cualquier número mostrado arriba. Los filtros de liga, split, fechas, parche y equipo son comunes a todas las vistas.

Versión navegable publicada en: <https://misstery13.github.io/bi-lol-datamart/dashboard/>

## 8. Matriz de trazabilidad

| Componente        | Pregunta / KPI                        | Filtros                     | Origen de datos                                         | Objeto SQL                                 |
|-------------------|---------------------------------------|-----------------------------|---------------------------------------------------------|--------------------------------------------|
| Tarjeta KPI 1     | Tasa de victoria del equipo           | Liga, split, fechas, equipo | hecho_partida_equipo + dim_equipo, dim_liga, dim_tiempo | fn_tasa_victoria()                         |
| Tarjeta KPI 2     | Diferencia de oro al minuto 15        | Liga, fechas                | hecho_partida_equipo + dim_liga, dim_tiempo             | vw_ventaja_temprana                        |
| Tarjeta KPI 3     | Presencia del campeón más contestado  | Parche, liga                | hecho_draft + dim_campeon, dim_parche                   | fn_top_campeones()                         |
| Gráfico de barras | ¿Qué equipos rinden mejor en la liga? | Liga, split, fechas         | hecho_partida_equipo + dim_equipo, dim_liga             | vw_kpi_equipo                              |
| Gráfico de línea  | ¿Cómo evoluciona la ventaja temprana? | Liga, fechas, equipo        | hecho_partida_equipo + dim_tiempo, dim_liga             | vw_ventaja_temprana, fn_evolucion_equipo() |
| Tabla de detalle  | ¿Qué campeones dominan el metajuego?  | Parche, liga, posición      | hecho_draft + dim_campeon, dim_parche                   | vw_presencia_campeon                       |
| Filtros globales  | Acotan todos los componentes          | —                           | dim_liga, dim_tiempo, dim_parche, dim_equipo            | SELECT DISTINCT sobre cada dimensión       |

## 9. Conclusión

¿Cómo almacenamos la información?

En un Data Mart en PostgreSQL con esquema estrella, compuesto por seis dimensiones y dos tablas de hechos, con la granularidad fijada en equipo y partida.

¿Cómo la vamos a consultar?

Mediante cuatro vistas que resuelven las agregaciones frecuentes y tres funciones parametrizadas que reciben los filtros del dashboard.

¿Cómo la vamos a presentar?

A través de una aplicación de cuatro vistas, cuyo diseño está definido en el mockup y cuyos componentes quedan trazados uno a uno con su objeto SQL de origen en la matriz de la sección anterior.